

Reviewer Pack — Minimal Rank of Symplectic Leaves Bounds DPLL Search Tree He...

Entry 1b6d2c2ffe68 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 23:25:10 UTC

Minimal Rank of Symplectic Leaves Bounds DPLL Search Tree Height

Entry ID: 1b6d2c2ffe68 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-28 23:25:10 UTC

1. Conjecture

Field A (mathematical branch): Symplectic Geometry **Field B** (complexity object): Complexity Theory: DPLL Search Tree Complexity

Statement:

[‘For a given symmetric matrix A , let $L(A)$ be the set of its symplectic leaves. The minimal rank of $L(A)$, denoted as $\kappa(L(A))$, is upper bounded by the height of the DPLL search tree for the corresponding SAT instance.’, ‘Formally, there exists a constant $c > 0$ such that for all symmetric matrices A , $\kappa(L(A)) \leq c \cdot h(A)$, where $h(A)$ is the height of the DPLL search tree for the SAT instance represented by matrix A .’, ‘For any given $m \times m$ symmetric matrix A and its corresponding $\kappa(L(A))$, if the bound $\kappa(L(A)) \leq c \cdot h(A)$ does not hold, then there exists a counterexample.’]

Rationale (proposer’s reasoning):

[‘Symplectic geometry has been previously applied to study the structure of real symmetric matrices, which are related to boolean functions used in complexity theory.’, ‘The symmetry of a matrix can be used to construct symplectic leaves, and these leaves could potentially provide structural information about the DPLL search tree height for SAT instances.’, ‘This conjecture aims to bridge the gap between symplectic geometry and complexity theory by proposing a quantitative relationship that could lead to new insights into the complexity of SAT.’]

Taxonomy category: GCT_DET_PERM (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 6159df356fc673a5

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if all generated matrices of size n up to 40×40 satisfy $\kappa(L(A)) \leq c \cdot h(A)$, where $\kappa(L(A))$ is the minimal rank of symplectic leaves and $h(A)$ is the DPLL search

tree height, with a support fraction ≥ 0.95 AND the average ratio of $\kappa(L(A))/c/h(A)$ across all matrices ≤ 1.05 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 4 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - symplectic geometry AND DPLL search tree - minimal rank symplectic leaves AND SAT instance complexity - DPLL tree height bound ON symplectic matrix

Top relevant hits considered: - [<http://arxiv.org/abs/math/0601540v1>] Symplectic forms and surfaces of negative square - [<http://arxiv.org/abs/1811.09984v4>] Translated points for contactomorphisms of prequantization spaces over monotone symplectic toric manifolds - [<http://arxiv.org/abs/1007.4036v2>] Symplectic reduction of quasi-morphisms and quasi-states - [<http://arxiv.org/abs/1210.1135v2>] The closure of the symplectic cone of elliptic surfaces

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=2.5s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        max_row = i + max(range(i, n), key=lambda j: abs(A[j][i]))
        A[i], A[max_row] = A[max_row], A[i]
        if A[i][i] == 0:
            continue
        denom = A[i][i]
        for j in range(n):
            A[i][j] /= denom
        for k in range(n):
            if k != i and A[k][i] != 0:
                factor = A[k][i]
                for j in range(n):
                    A[k][j] -= factor * A[i][j]
    return A

def matrix_rank(A):
```

```

n = len(A)
rank = 0
for i in range(n):
    if all(abs(A[j][i]) < 1e-9 for j in range(rank)):
        continue
    rank += 1
    denom = A[i][i]
    for j in range(n):
        A[i][j] /= denom
    for k in range(n):
        if k != i and abs(A[k][i]) > 1e-9:
            factor = A[k][i]
            for j in range(n):
                A[k][j] -= factor * A[i][j]
return rank

def convert_matrix_to_cnf(A):
    n = len(A)
    cnf = []
    for i in range(n):
        for j in range(i + 1, n):
            if A[i][j] != 0:
                clause = [i * n + j + 1, -(i * n + i + 1), -(j * n + j + 1)]
                cnf.append(clause)
    return cnf

def dpll_search_tree_height(cnf):
    def dpll(assignment):
        if not cnf:
            return 0
        unit_clauses = [c for c in cnf if len(c) == 1]
        if unit_clauses:
            literal = unit_clauses[0][0]
            new_assignment = assignment[:]
            new_assignment[abs(literal) - 1] = literal > 0
            return dpll(new_assignment)
        pure_literals = [l for l in range(1, n * n + 1) if (all(l not in c for c in cnf) or all(-l not in c for c in cnf))]
        if pure_literals:
            literal = pure_literals[0]
            new_assignment = assignment[:]
            new_assignment[abs(literal) - 1] = literal > 0
            return dpll(new_assignment)
        literal = random.choice([l for l in range(1, n * n + 1)])
        new_assignment_true = assignment[:]
        new_assignment_true[abs(literal) - 1] = literal > 0
        height_true = dpll(new_assignment_true)
        if height_true == float('inf'):
            return float('inf')
        new_assignment_false = assignment[:]
        new_assignment_false[abs(literal) - 1] = literal < 0
        height_false = dpll(new_assignment_false)
        if height_false == float('inf'):
            return float('inf')
        return max(height_true, height_false) + 1
    n = int(math.sqrt(len(cnf)))
    assignment = [False] * (n * n)

```

```

    return dpll(assignment)

def symplectic_leaves_rank(A):
    # Placeholder for the actual implementation of symplectic leaves rank calculation
    return matrix_rank(gaussian_elimination(A))

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.choice([5, 10, 15, 20, 30, 40])
    A = [[random.randint(-10, 10) for _ in range(n)] for _ in range(n)]
    for i in range(n):
        for j in range(i + 1, n):
            A[j][i] = A[i][j]
    cnf = convert_matrix_to_cnf(A)
    h_A = dpll_search_tree_height(cnf)
    kappa_L_A = symplectic_leaves_rank(A)
    if h_A == float('inf'):
        return {
            "metric_name": "kappa_L_A / c * h_A",
            "metric_value": None,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "DPLL search tree height is infinite"
        }
    if kappa_L_A > 1000: # Arbitrary large number to avoid trivial cases
        return {
            "metric_name": "kappa_L_A / c * h_A",
            "metric_value": None,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "Symplectic leaves rank is too large"
        }
    if kappa_L_A <= 0:
        return {
            "metric_name": "kappa_L_A / c * h_A",
            "metric_value": None,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "Symplectic leaves rank is non-positive"
        }
    c = Fraction(1, 2) # Placeholder for the actual constant
    return {
        "metric_name": "kappa_L_A / c * h_A",
        "metric_value": kappa_L_A / (c * h_A),
        "instances_tested": 1,
        "conjecture_holds": kappa_L_A <= c * h_A,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        results.append(trial_result)

```

```

print(f"TRIAL: {trial_result}")

if all(result["conjecture_holds"] for result in results):
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)
    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value) ** 2 for result in results) / len(results))
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif any(not result["conjecture_holds"] for result in results):
    first_failing_seed = next((seed for seed, result in zip(seeds, results) if not result["conjecture_holds"]), None)
    print(f"RESULT: FALSIFIED counterexample=\"kappa_L_A > c * h_A\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5e19c58d.py", line 146, in <module>
    trial_result = run_trial(seed)
                    ~~~~~^~~~~~

  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5e19c58d.py", line 106, in run_trial
    h_A = dpll_search_tree_height(cnf)
           ~~~~~^~~~~~

  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5e19c58d.py", line 92, in dpll_search_tree_height
    return dpll(assignment)
           ~~~~~^~~~~~

  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5e19c58d.py", line 77, in dpll
    return dpll(new_assignment)
           ~~~~~^~~~~~

  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5e19c58d.py", line 77, in dpll
    return dpll(new_assignment)
           ~~~~~^~~~~~

  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5e19c58d.py", line 77, in dpll
    return dpll(new_assignment)
           ~~~~~^~~~~~

  [Previous line repeated 993 more times]
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5e19c58d.py", line 72, in dpll
    pure_literals = [l for l in range(1, n * n + 1) if (all(l not in c for c in cnf) or all(
l not in c for c in cnf))]
                                                                    ~~~~~^~~~~~

RecursionError: maximum recursion depth exceeded

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing any data, which means that the pre-registered support condition could not be unambiguously met. | next: Re-run the test with increased recursion limits or use a different approach to avoid recursion errors.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12561
2	preregistration	ollama_remote	glm4:latest	0	0	6218
3	novelty	ollama_remote	glm4:latest	0	0	4540
4	novelty	ollama_remote	glm4:latest	0	0	5962
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20864
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10598
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11110
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18425
9	judge	ollama_remote	glm4:latest	0	0	8072

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 98350 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/1b6d2c2ffe68.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/1b6d2c2ffe68.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/1b6d2c2ffe68.tar.gz (if generated)